

DataLab 实验报告

姓名：刘韬 学号：2025201694

实验结果

函数	得分	函数	得分
bitAnd	1/1	bitXor	1/1
samesign	2/2	logtwo	4/4
byteSwap	4/4	reverse	3/3
logicalShift	3/3	leftBitCount	4/4
float_i2f	4/4	floatScale2	4/4
float64_f2i	3/3	floatPower2	4/4
总分	37/37	测试结论	全部通过

所有函数的正确性、合法运算符和最大操作次数检查均通过。

```

●●● Ubuntu - DataLab test
$ python3 test.py
Make success.
bitAnd      1/1:    PASS
bitXor      1/1:    PASS
samesign    2/2:    PASS
logtwo      4/4:    PASS
byteSwap    4/4:    PASS
reverse     3/3:    PASS
logicalShift 3/3:    PASS
leftBitCount 4/4:    PASS
float_i2f    4/4:    PASS
floatScale2  4/4:    PASS
float64_f2i  3/3:    PASS
floatPower2  4/4:    PASS
Total points: 37
```

解题思路

bitAnd

利用德摩根律，把按位与转换为按位取反和按位或：分别取反后按位或，再对整体取反。

bitXor

异或表示对应位不同。通过排除两个数同为 1 和同为 0 的情况，只使用 ~ 与 & 构造结果。

samesign

先单独处理 0；对于两个非零整数，比较最高符号位是否相同。

logtwo

最高位 1 的位置就是向下取整的二进制对数。分段检查高 16、8、4、2、1 位来快速定位。

byteSwap

取出两个目标字节，计算它们的异或差异掩码，再把掩码移回两个位置并作用到原数上。

reverse

循环 32 次，每次取最低位追加到结果中，然后将输入右移一位。

logicalShift

先执行右移，再构造高位为 0 的掩码，清除算术右移对负数补入的 1。

leftBitCount

依次判断最高端的 16、8、4、2、1 位是否全为 1，并根据结果移位和累加计数。

float_i2f

提取符号并计算绝对值，最高位决定指数，其余部分组成尾数；截断时按 IEEE 754 的舍入到偶数规则处理。

floatScale2

分别处理 NaN/无穷大、非规格化数和规格化数：移动有效位或增加指数，并处理溢出。

float64_f2i

从高 32 位提取符号和指数，再组合高、低尾数得到整数有效位；下溢返回 0，溢出返回 0x80000000。

floatPower2

根据 x 的范围分别构造 0、非规格化数、规格化数或正无穷。

亮点

1. logtwo 与 leftBitCount 使用分段判断，避免逐位扫描。
2. float_i2f 正确处理舍入到偶数和尾数进位。
3. 全部函数均满足 Legal ops 和 Max ops 限制。

收获与总结

本实验加深了我对补码、符号位、逻辑移位和算术移位的理解，也让我熟悉了 IEEE 754 浮点数中符号、指数和尾数的组织方式。限制运算符和操作次数后，需要直接从二进制表示出发设计算法。

参考资料

1. 课程提供的 DataLab README 与测试程序。
2. Randal E. Bryant、David R. O'Hallaron, 《深入理解计算机系统》，数据表示相关章节。